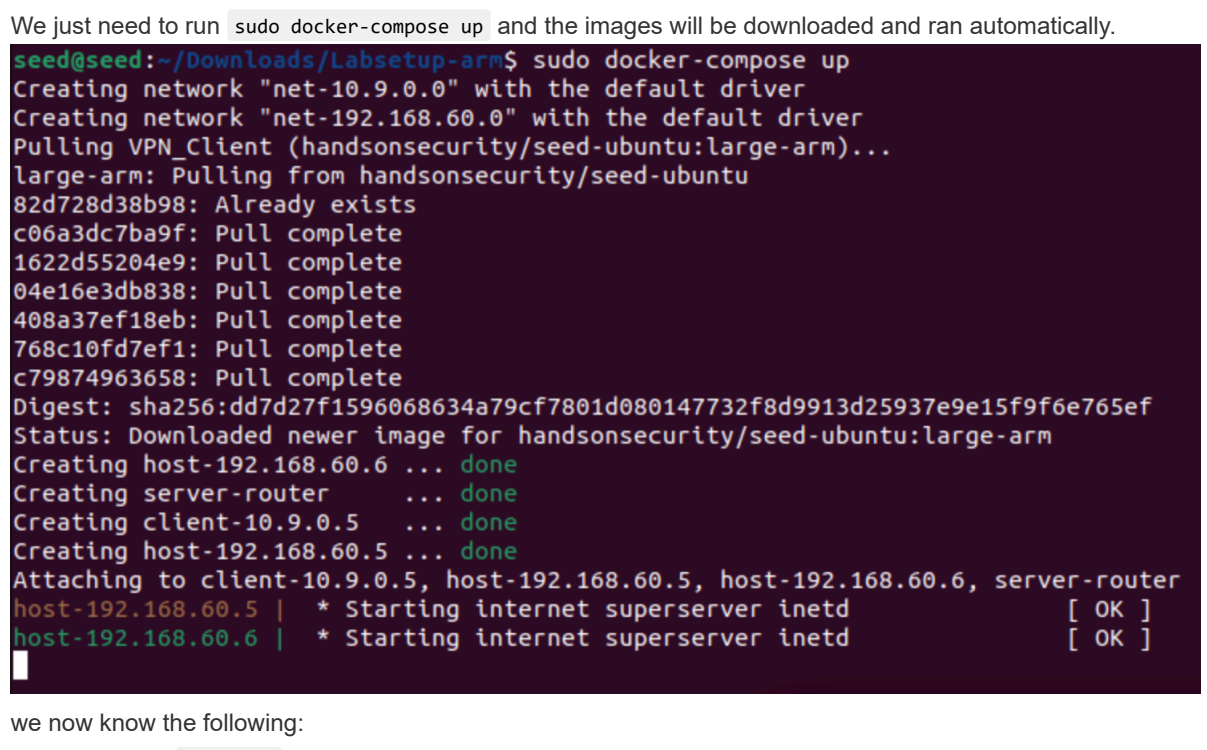
50.020 Network Security Lab 7 VPN

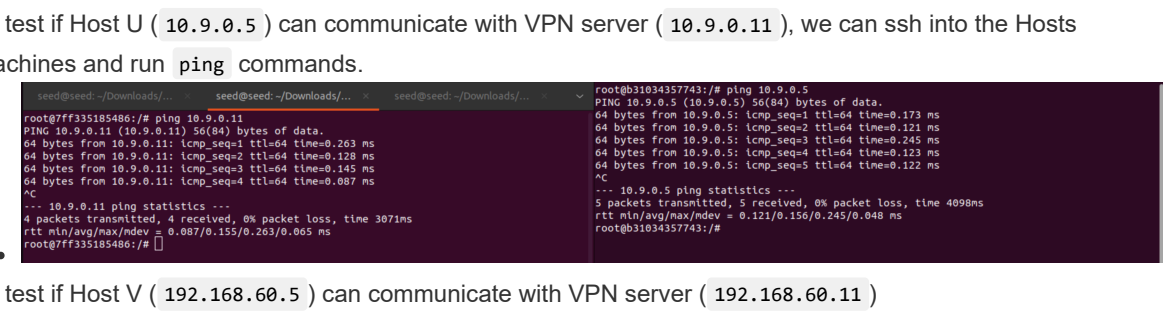
Task 1: Setting up the Lab Environment

1. The lab zip file uses images, hence there is no need to build them from Dockerfiles.
   - We just need to run `sudo docker-compose up` and the images will be downloaded and ran automatically.



```
seed@seed:~/Downloads/Labsetup-arm$ sudo docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
Pulling VPN_Client (handsonsecurity/seed-ubuntu:large-arm)...
large-arm: Pulling from handsonsecurity/seed-ubuntu
82d728d38b98: Already exists
c06a3dc7ba9f: Pull complete
1622d55204e9: Pull complete
04e16e3db838: Pull complete
408a37ef18eb: Pull complete
768c10fd7ef1: Pull complete
c79874963658: Pull complete
Digest: sha256:dd7d27f1596068634a79cf7801d080147732f8d9913d25937e9e15f9f6e765ef
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:large-arm
Creating host-192.168.60.6 ... done
Creating server-router     ... done
Creating client-10.9.0.5   ... done
Creating host-192.168.60.5 ... done
Attaching to client-10.9.0.5, host-192.168.60.5, host-192.168.60.6, server-router
host-192.168.60.5 |  * Starting internet superserver inetd                  [ OK ]
host-192.168.60.6 |  * Starting internet superserver inetd                  [ OK ]
```

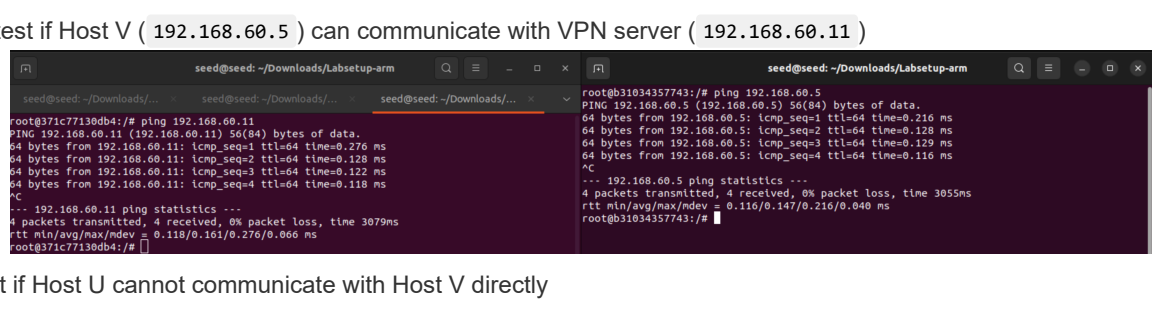
   - we now know the following:
     - Host U IP: `10.9.0.5`
     - Host V IP: `192.168.60.5`
     - VPN Server IP `10.9.0.11` (external ip), `192.168.60.11` (internal ip)
2. Then we run the tests instructed by the lab instructions to test if the setup was correct.
   i. To test if Host U (`10.9.0.5`) can communicate with VPN server (`10.9.0.11`), we can ssh into the Hosts machines and run `ping` commands.



```
root@7ff335185486:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.263 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.128 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.145 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.087 ms
^C
--- 10.9.0.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3071ms
rtt min/avg/max/mdev = 0.087/0.155/0.263/0.065 ms
root@7ff335185486:/#
```

```
root@b31034357743:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=64 time=0.173 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=64 time=0.121 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=64 time=0.245 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=64 time=0.123 ms
64 bytes from 10.9.0.5: icmp_seq=5 ttl=64 time=0.122 ms
^C
--- 10.9.0.5 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4098ms
rtt min/avg/max/mdev = 0.121/0.156/0.245/0.048 ms
root@b31034357743:/#
```

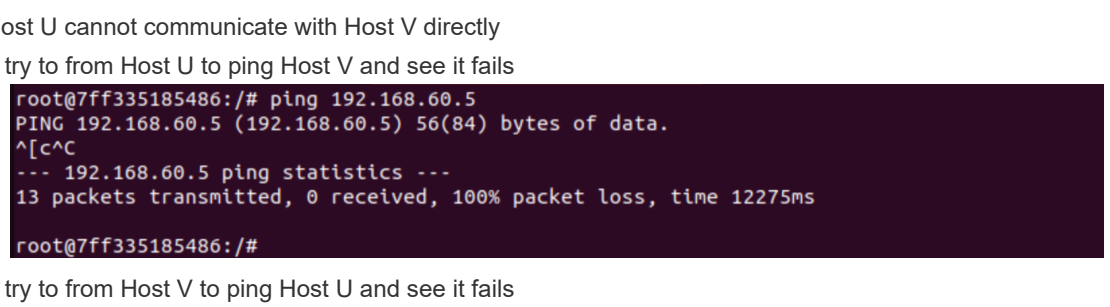
   ii. To test if Host V (`192.168.60.5`) can communicate with VPN server (`192.168.60.11`)



```
root@371c77130db4:/# ping 192.168.60.11
PING 192.168.60.11 (192.168.60.11) 56(84) bytes of data.
64 bytes from 192.168.60.11: icmp_seq=1 ttl=64 time=0.276 ms
64 bytes from 192.168.60.11: icmp_seq=2 ttl=64 time=0.128 ms
64 bytes from 192.168.60.11: icmp_seq=3 ttl=64 time=0.122 ms
64 bytes from 192.168.60.11: icmp_seq=4 ttl=64 time=0.118 ms
^C
--- 192.168.60.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3079ms
rtt min/avg/max/mdev = 0.118/0.161/0.276/0.066 ms
root@371c77130db4:/#
```

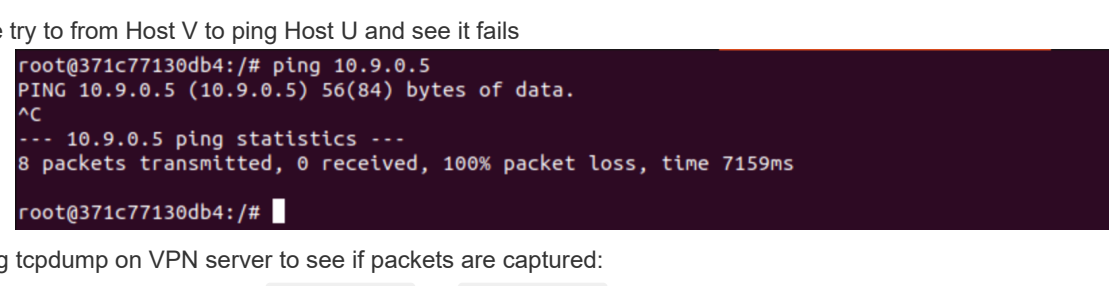
```
root@b31034357743:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=64 time=0.216 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=64 time=0.128 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=64 time=0.129 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=64 time=0.116 ms
^C
--- 192.168.60.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.116/0.147/0.216/0.040 ms
root@b31034357743:/#
```

   iii. Test if Host U cannot communicate with Host V directly
   - We try to from Host U to ping Host V and see it fails



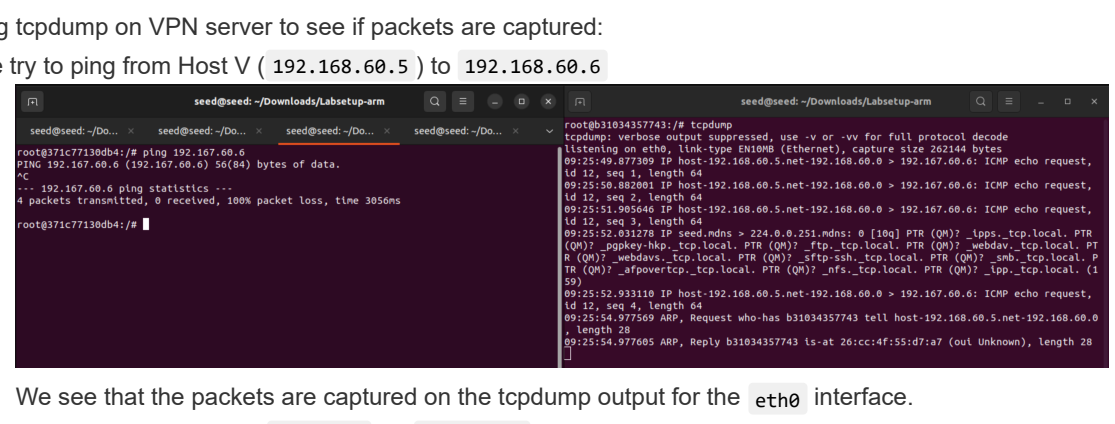
```
root@7ff335185486:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^[c^C
--- 192.168.60.5 ping statistics ---
13 packets transmitted, 0 received, 100% packet loss, time 12275ms

root@7ff335185486:/#
```

   - We try to from Host V to ping Host U and see it fails



```
root@371c77130db4:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
^C
--- 10.9.0.5 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7159ms

root@371c77130db4:/#
```

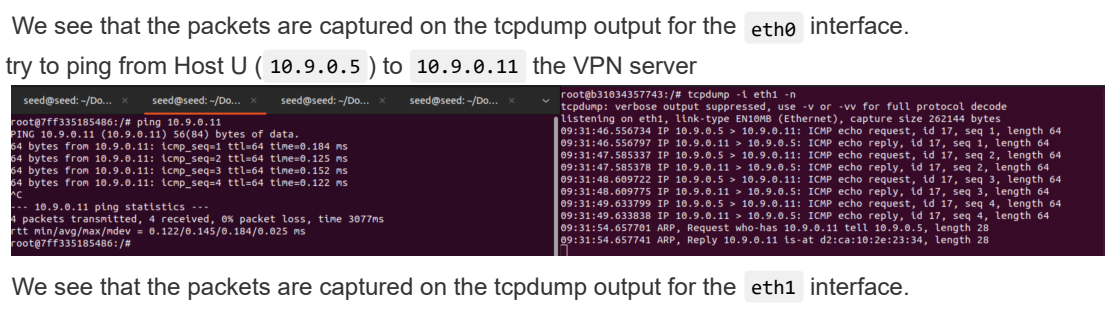
   iv. Running tcpdump on VPN server to see if packets are captured:
   - We try to ping from Host V (`192.168.60.5`) to `192.168.60.6`



```
root@371c77130db4:/# ping 192.167.60.6
PING 192.167.60.6 (192.167.60.6) 56(84) bytes of data.
^C
--- 192.167.60.6 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3056ms

root@371c77130db4:/#
```

```
root@b31034357743:/# tcpdump
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
09:25:49.877309 IP host-192.168.60.5.net-192.168.60.0 > 192.167.60.6: ICMP echo request, id 12, seq 1, length 64
09:25:50.882001 IP host-192.168.60.5.net-192.168.60.0 > 192.167.60.6: ICMP echo request, id 12, seq 2, length 64
09:25:51.905646 IP host-192.168.60.5.net-192.168.60.0 > 192.167.60.6: ICMP echo request, id 12, seq 3, length 64
09:25:52.031278 IP seed.mdns > 224.0.0.251.mdns: 0 [10q] PTR (QM)? _ipps._tcp.local. PTR (QM)? _pgpkey-hkp._tcp.local. PTR (QM)? _ftp._tcp.local. PTR (QM)? _webdav._tcp.local. PTR (QM)? _webdavs._tcp.local. PTR (QM)? _sftp-ssh._tcp.local. PTR (QM)? _smb._tcp.local. PTR (QM)? _afpovertcp._tcp.local. PTR (QM)? _nfs._tcp.local. PTR (QM)? _ipp._tcp.local. (159)
09:25:52.933110 IP host-192.168.60.5.net-192.168.60.0 > 192.167.60.6: ICMP echo request, id 12, seq 4, length 64
09:25:54.977569 ARP, Request who-has b31034357743 tell host-192.168.60.5.net-192.168.60.0, length 28
09:25:54.977605 ARP, Reply b31034357743 is-at 26:cc:4f:55:d7:a7 (oui Unknown), length 28
```

     - We see that the packets are captured on the tcpdump output for the `eth0` interface.
   - We try to ping from Host U (`10.9.0.5`) to `10.9.0.11` the VPN server



```
root@7ff335185486:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.184 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.125 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.152 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.122 ms
^C
--- 10.9.0.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3077ms
rtt min/avg/max/mdev = 0.122/0.145/0.184/0.025 ms
root@7ff335185486:/#
```

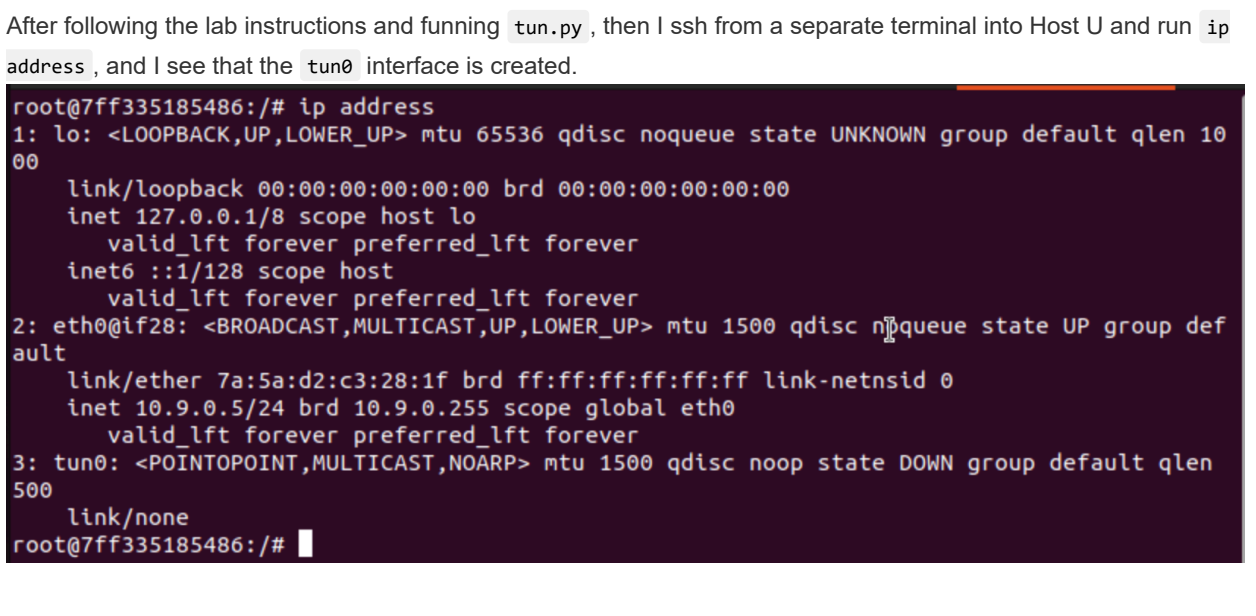
```
root@b31034357743:/# tcpdump -i eth1 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
09:31:46.556734 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 17, seq 1, length 64
09:31:46.556797 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 17, seq 1, length 64
09:31:47.585337 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 17, seq 2, length 64
09:31:47.585378 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 17, seq 2, length 64
09:31:48.609722 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 17, seq 3, length 64
09:31:48.609775 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 17, seq 3, length 64
09:31:49.633799 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 17, seq 4, length 64
09:31:49.633838 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 17, seq 4, length 64
09:31:54.657701 ARP, Request who-has 10.9.0.11 tell 10.9.0.5, length 28
09:31:54.657741 ARP, Reply 10.9.0.11 is-at d2:ca:10:2e:23:34, length 28
```

     - We see that the packets are captured on the tcpdump output for the `eth1` interface.

Task 2: Create and configure TUN interface

Task 2.a: Create TUN interface on Host U

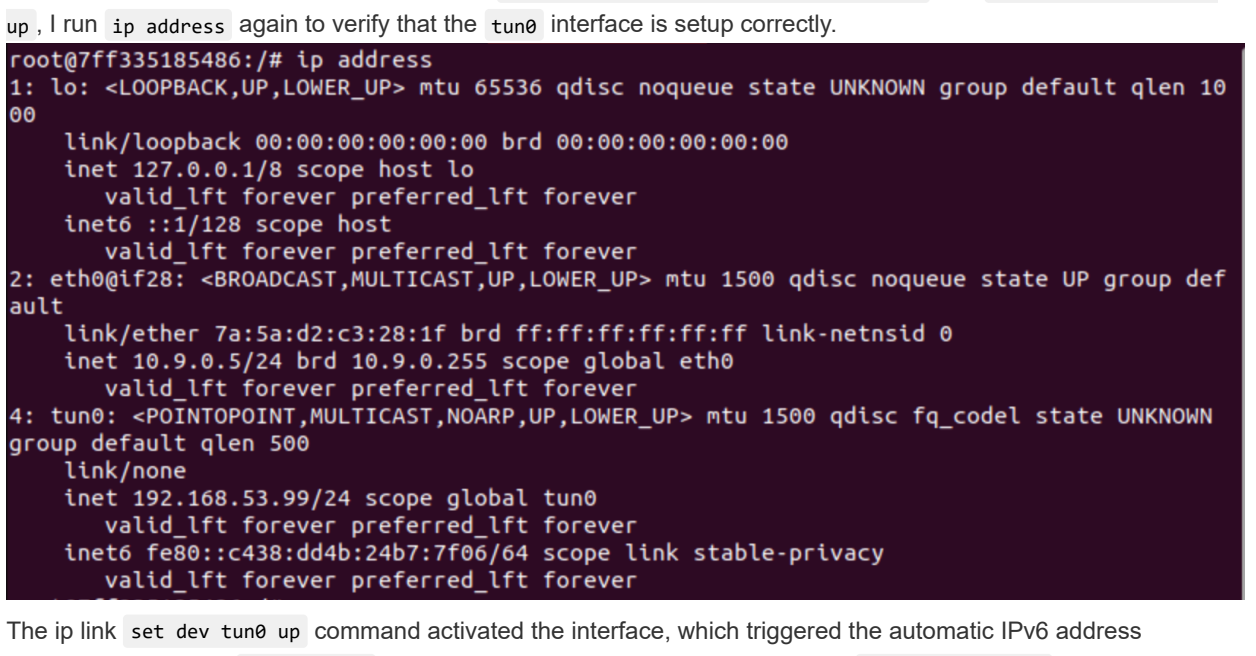
- After following the lab instructions and funning `tun.py`, then I ssh from a separate terminal into Host U and run `ip address`, and I see that the `tun0` interface is created.



```
root@7ff335185486:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 10
00
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if28: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group def
ault
    link/ether 7a:5a:d2:c3:28:1f brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN group default qlen
500
    link/none
root@7ff335185486:/#
```

Task 2.b: Setup the TUN interface

- After following the lab instruction and running `ip addr add 192.168.53.99/24 dev tun0` and `ip link set dev tun0 up`, I run `ip address` again to verify that the `tun0` interface is setup correctly.



```
root@7ff335185486:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 10
00
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if28: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group def
ault
    link/ether 7a:5a:d2:c3:28:1f brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
4: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN
group default qlen 500
    link/none
    inet 192.168.53.99/24 scope global tun0
       valid_lft forever preferred_lft forever
    inet6 fe80::c438:dd4b:24b7:7f06/64 scope link stable-privacy
       valid_lft forever preferred_lft forever
```

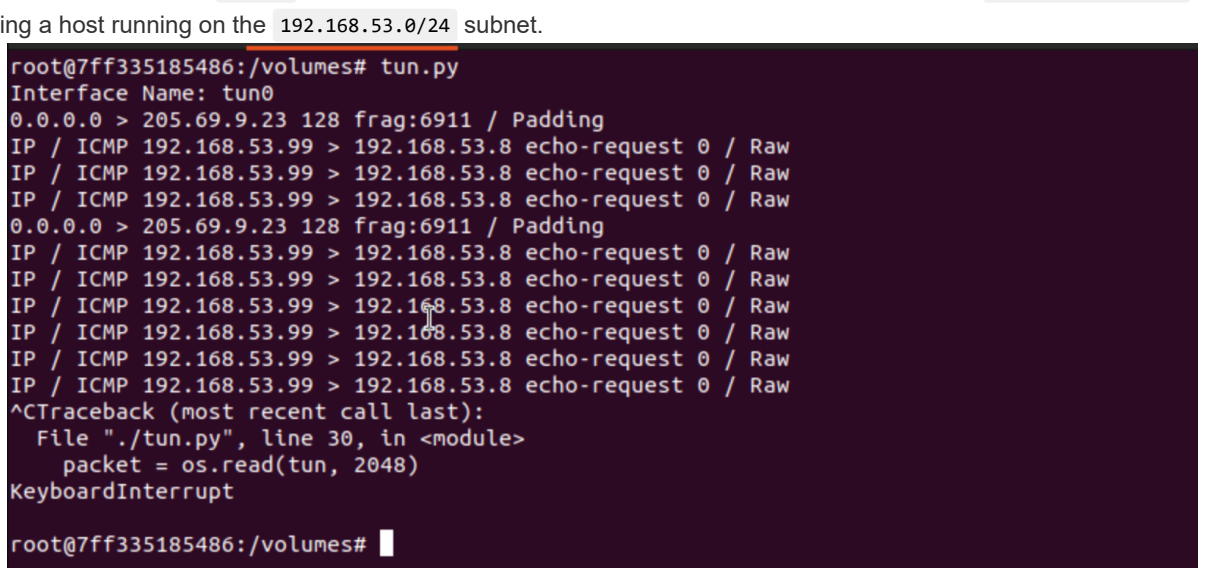
- The ip link `set dev tun0 up` command activated the interface, which triggered the automatic IPv6 address generation, while the `ip addr add` command explicitly added the IPv4 address `192.168.53.99/24`.

Task 2.c: Read from the TUN interface

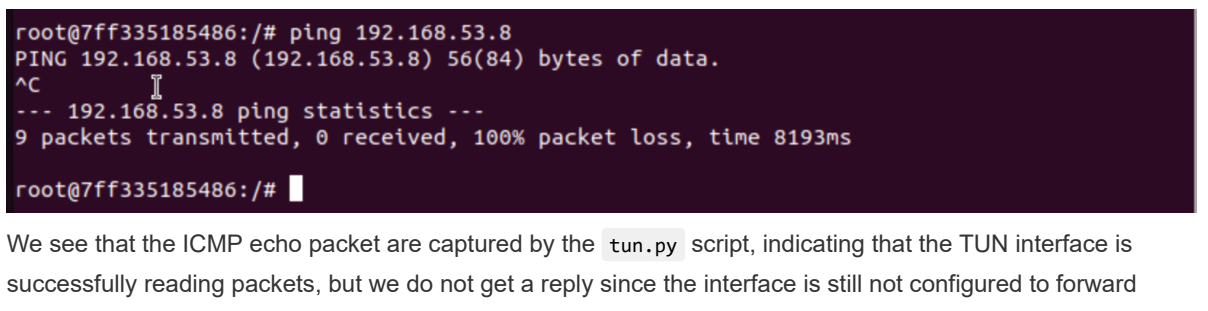
- By following the lab instruction i edit the `tun.pt` file to now contain this python code

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
print("Interface Name: {}".format(ifname))

# used to up the TUN interface
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        ip = IP(packet)
        print(ip.summary())
```

1. Then I run the modified `tun.py` on Host U, and from another terminal I ssh into Host U and run `ping 192.168.53.8` to ping a host running on the `192.168.53.0/24` subnet.

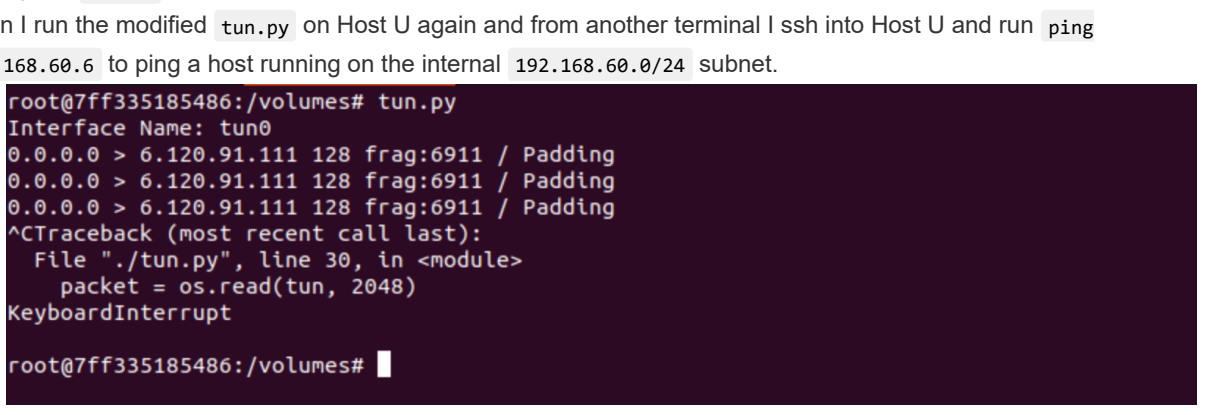


```
root@7ff335185486:/volumes# tun.py
Interface Name: tun0
0.0.0.0 > 205.69.9.23 128 frag:6911 / Padding
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
0.0.0.0 > 205.69.9.23 128 frag:6911 / Padding
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
^CTraceback (most recent call last):
  File "./tun.py", line 30, in <module>
    packet = os.read(tun, 2048)
KeyboardInterrupt

root@7ff335185486:/volumes#
```

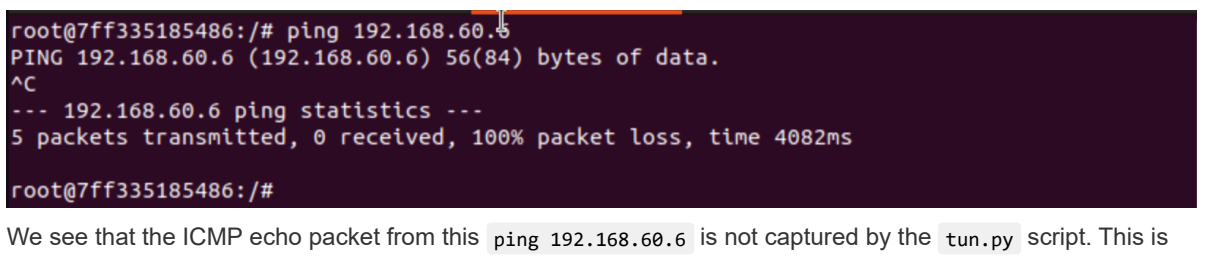


```
root@7ff335185486:/# ping 192.168.53.8
PING 192.168.53.8 (192.168.53.8) 56(84) bytes of data.
^C
--- 192.168.53.8 ping statistics ---
9 packets transmitted, 0 received, 100% packet loss, time 8193ms

root@7ff335185486:/#
```

   - We see that the ICMP echo packet are captured by the `tun.py` script, indicating that the TUN interface is successfully reading packets, but we do not get a reply since the interface is still not configured to forward packets, however since they are on the same LAN network, the packets are still sent out, which are captured by the `tun.py` script.
2. Then I run the modified `tun.py` on Host U again and from another terminal I ssh into Host U and run `ping 192.168.60.6` to ping a host running on the internal `192.168.60.0/24` subnet.



```
root@7ff335185486:/volumes# tun.py
Interface Name: tun0
0.0.0.0 > 6.120.91.111 128 frag:6911 / Padding
0.0.0.0 > 6.120.91.111 128 frag:6911 / Padding
0.0.0.0 > 6.120.91.111 128 frag:6911 / Padding
^CTraceback (most recent call last):
  File "./tun.py", line 30, in <module>
    packet = os.read(tun, 2048)
KeyboardInterrupt

root@7ff335185486:/volumes#
```



```
root@7ff335185486:/# ping 192.168.60.6
PING 192.168.60.6 (192.168.60.6) 56(84) bytes of data.
^C
--- 192.168.60.6 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4082ms

root@7ff335185486:/#
```

   - We see that the ICMP echo packet from this `ping 192.168.60.6` is not captured by the `tun.py` script. This is because the host U trying to send packets to a network outside of its LAN network, hence no packets are sent out as the network is unreachable.
   - Therefore we see that `tun.py` does not capture any ICMP echo packets as no packets are sent out from the TUN interface.

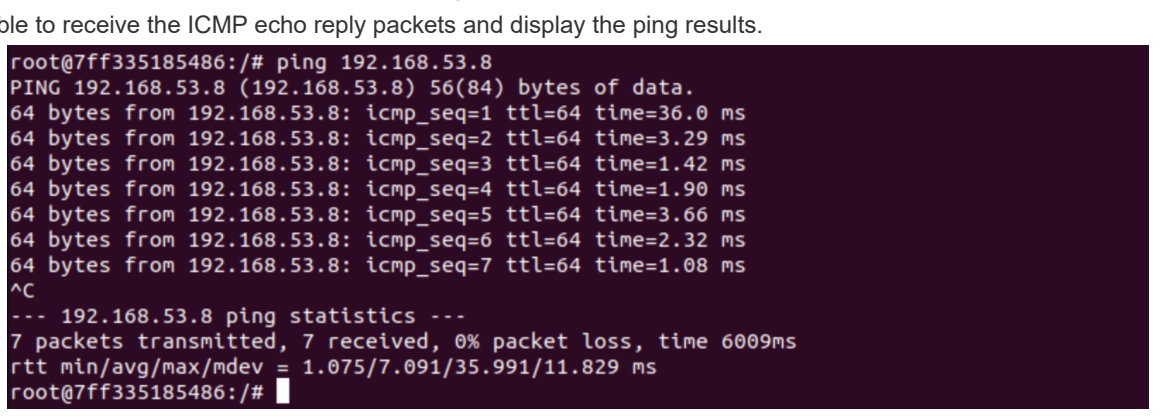
Task 2.d: Write to the TUN interface

- Following the requirements set in the lab instructions, I modified the `tun.py` file to execute the 2 following requirements:
  - If receive an ICMP echo packet, construct an ICMP echo reply packet and write it back to the TUN interface.
  - Instead of writing a IP packet to the TUN interface, write some arbitrary data to the TUN interface.

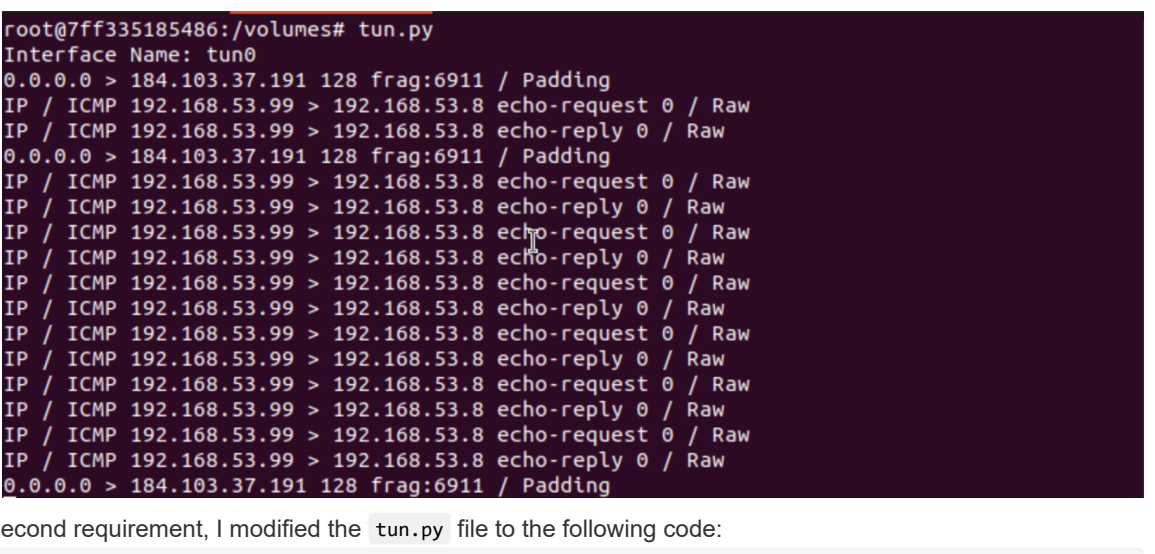
1. For the first requirement, I modified the `tun.py` file to the following code:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
print("Interface Name: {}".format(ifname))

# used to up the TUN interface
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        ip = IP(packet)
        print(ip.summary())

        # Check if the packet is an ICMP echo request
        if ip.haslayer(ICMP) and ip[ICMP].type == 8:
            # Construct ICMP echo reply packet from
            icmp_reply = IP(src =ip.dst, dst = ip.src)
            icmp_reply_ptkt = icmp_reply/ip.payload
            os.write(tun, bytes(icmp_reply_ptkt))
```

   - Then now when I run `ping 192.168.53.8` on Host U, I see that the `tun.py` script captures the ICMP echo request packets and construct ICMP echo reply packets and write them back to the TUN interface, and Host U is able to receive the ICMP echo reply packets and display the ping results.



```
root@7ff335185486:/# ping 192.168.53.8
PING 192.168.53.8 (192.168.53.8) 56(84) bytes of data.
64 bytes from 192.168.53.8: icmp_seq=1 ttl=64 time=36.0 ms
64 bytes from 192.168.53.8: icmp_seq=2 ttl=64 time=3.29 ms
64 bytes from 192.168.53.8: icmp_seq=3 ttl=64 time=1.42 ms
64 bytes from 192.168.53.8: icmp_seq=4 ttl=64 time=1.90 ms
64 bytes from 192.168.53.8: icmp_seq=5 ttl=64 time=3.66 ms
64 bytes from 192.168.53.8: icmp_seq=6 ttl=64 time=2.32 ms
64 bytes from 192.168.53.8: icmp_seq=7 ttl=64 time=1.08 ms
^C
--- 192.168.53.8 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6009ms
rtt min/avg/max/mdev = 1.075/7.091/35.991/11.829 ms
root@7ff335185486:/#
```

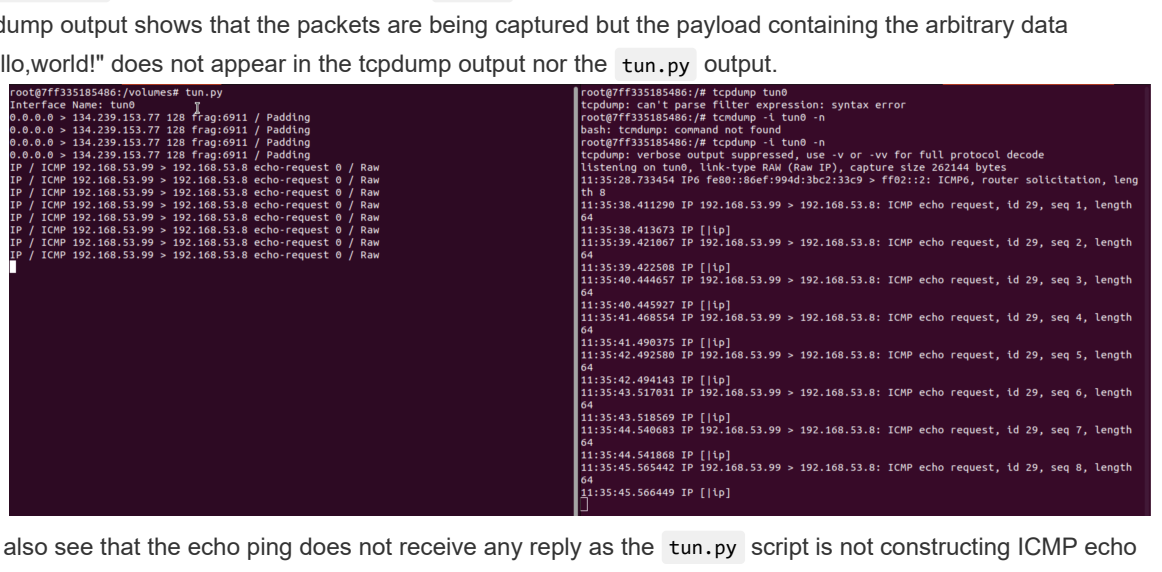


```
root@7ff335185486:/volumes# tun.py
Interface Name: tun0
0.0.0.0 > 184.103.37.191 128 frag:6911 / Padding
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
0.0.0.0 > 184.103.37.191 128 frag:6911 / Padding
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-reply 0 / Raw
0.0.0.0 > 184.103.37.191 128 frag:6911 / Padding
```

2. For the second requirement, I modified the `tun.py` file to the following code:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
print("Interface Name: {}".format(ifname))

# used to up the TUN interface
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        ip = IP(packet)
        print(ip.summary())

        if ICMP in ip:
            os.write(tun, bytes("Hello,world!", encoding='utf-8'))
```

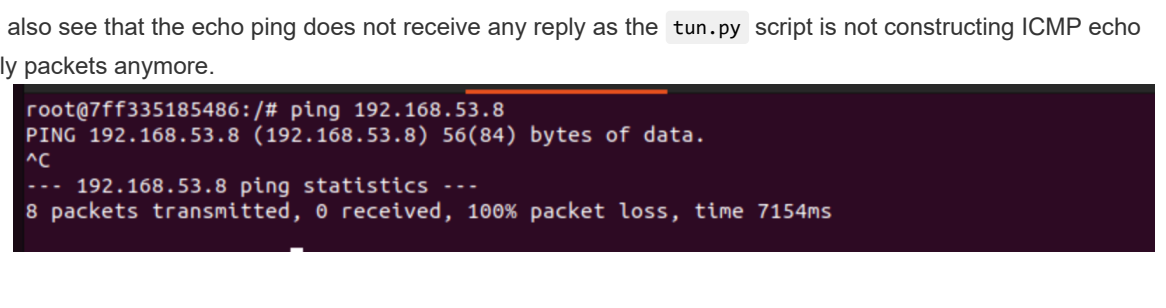
   - Then we run `tun.py` again, then start a `tcpdump -i tun0 -n` on a separate Host U terminal, and ping `192.168.53.8` from Host U, we see that the `tun.py` script captures the ICMP echo request packets, then the tcpdump output shows that the packets are being captured but the payload containing the arbitrary data "Hello,world!" does not appear in the tcpdump output nor the `tun.py` output.



```
root@7ff335185486:/volumes# tun.py
Interface Name: tun0
0.0.0.0 > 134.239.153.77 128 frag:6911 / Padding
0.0.0.0 > 134.239.153.77 128 frag:6911 / Padding
0.0.0.0 > 134.239.153.77 128 frag:6911 / Padding
0.0.0.0 > 134.239.153.77 128 frag:6911 / Padding
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.8 echo-request 0 / Raw
```

```
root@7ff335185486:/# tcpdump tun0
tcpdump: can't parse filter expression: syntax error
root@7ff335185486:/# tcmdump -i tun0 -n
bash: tcmdump: command not found
root@7ff335185486:/# tcpdump -i tun0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
11:35:28.733454 IP6 fe80::80ef:994d:3bc2:33c9 > ff02::2: ICMP6, router solicitation, length 8
11:35:38.411290 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 1, length 64
11:35:38.413673 IP [|ip]
11:35:39.421067 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 2, length 64
11:35:39.422588 IP [|ip]
11:35:40.444657 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 3, length 64
11:35:40.445927 IP [|ip]
11:35:41.468554 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 4, length 64
11:35:41.490375 IP [|ip]
11:35:42.492588 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 5, length 64
11:35:42.494143 IP [|ip]
11:35:43.517031 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 6, length 64
11:35:43.518569 IP [|ip]
11:35:44.540603 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 7, length 64
11:35:44.541608 IP [|ip]
11:35:45.565442 IP 192.168.53.99 > 192.168.53.8: ICMP echo request, id 29, seq 8, length 64
11:35:45.566449 IP [|ip]
```

   - We also see that the echo ping does not receive any reply as the `tun.py` script is not constructing ICMP echo reply packets anymore.



```
root@7ff335185486:/# ping 192.168.53.8
PING 192.168.53.8 (192.168.53.8) 56(84) bytes of data.
^C
--- 192.168.53.8 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7154ms
```

Task 3: Send the IP packets through to VPN server through Tunnel

- Following the instruction from the lab manual, I modified the `tun_client.py` and `tun_server.py` files to contain the following code:

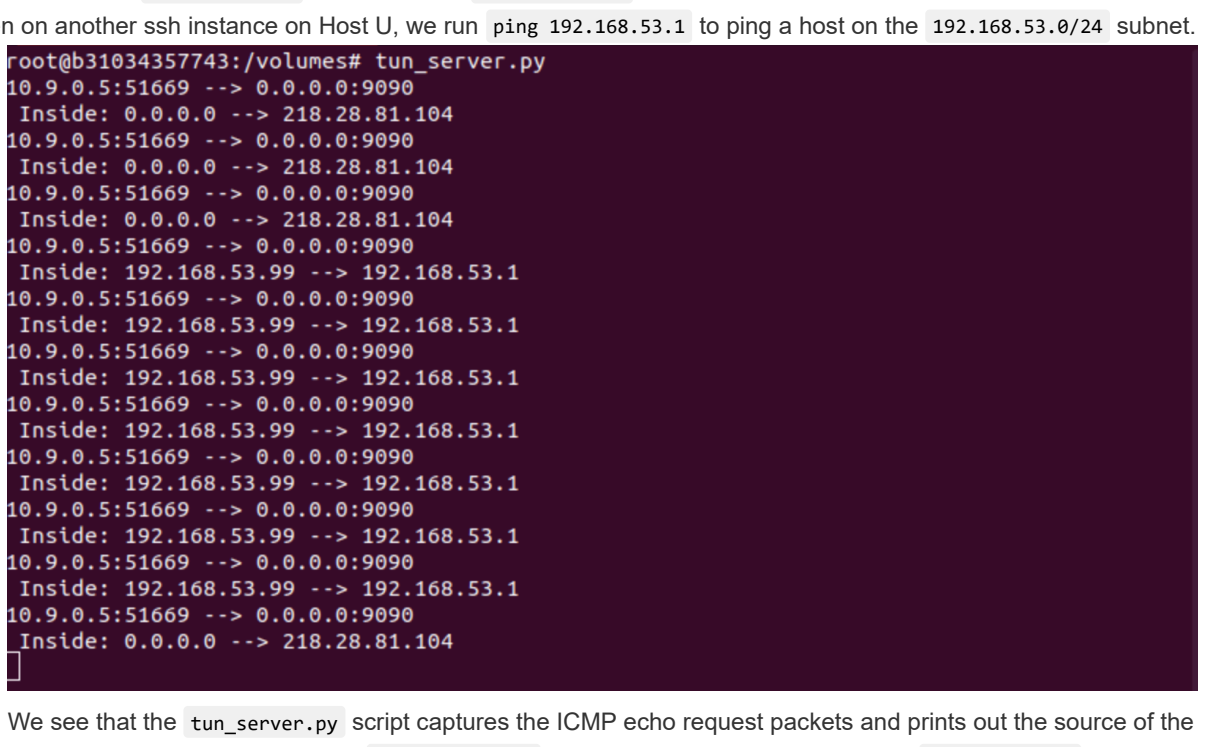
1. `tun_client.py`:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
SERVER_IP, SERVER_PORT = '10.9.0.11', 9090
while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        # Send the packet via the tunnel
        sock.sendto(packet, (SERVER_IP, SERVER_PORT))
```

2. `tun_server.py`:

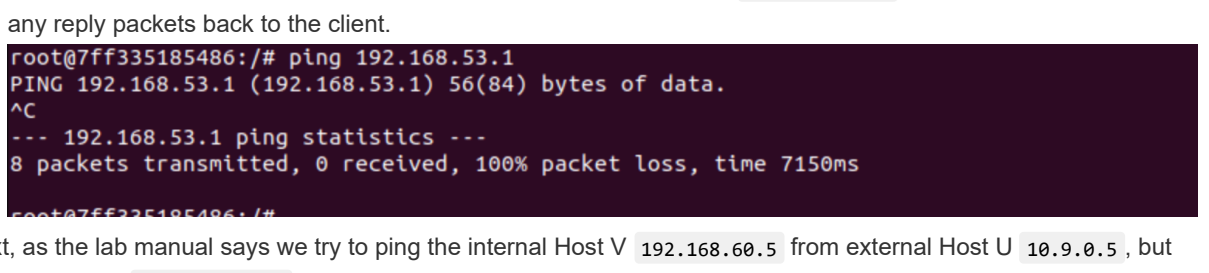
```python
#!/usr/bin/env python3

from scapy.all import *

IP_A = "0.0.0.0"
PORT = 9090

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

while True:
    data, (ip, port) = sock.recvfrom(2048)
    print("{}:{} --> {}:{}".format(ip, port, IP_A, PORT))
    pkt = IP(data)
    print(" Inside: {} --> {}".format(pkt.src, pkt.dst))
```

3. Then we run the `tun_client.py` on Host U and `tun_server.py` on the VPN server.
4. Then on another ssh instance on Host U, we run `ping 192.168.53.1` to ping a host on the `192.168.53.0/24` subnet.



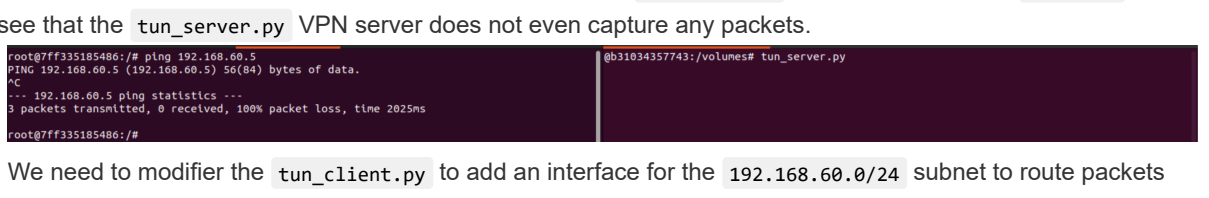
```
root@b31034357743:/volumes# tun_server.py
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 218.28.81.104
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 218.28.81.104
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 218.28.81.104
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:51669 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 218.28.81.104
```

   - We see that the `tun_server.py` script captures the ICMP echo request packets and prints out the source of the packets, from Host U and its port `10.9.0.5:51669` and the IP address and port of the `tun_server.py` on the VPN server on the UDP port `0.0.0.0:9090`.
   - We also see that the packet is transmitted through the internal interface from `192.168.53.99` to `192.168.53.1` since we have set up the tun0 interface on the VPN server, when the client tries to ping a host in the `192.168.53.0/24` subnet, the packet is routed through the tun0 interface on the VPN server `192.168.53.99` to reach the destination host `192.168.53.1`.
   - However, we do not receive any ICMP echo reply packets since the `tun_server.py` is not configured to send any reply packets back to the client.



```
root@7ff335185486:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
^C
--- 192.168.53.1 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7150ms
```

5. Next, as the lab manual says we try to ping the internal Host V `192.168.60.5` from external Host U `10.9.0.5`, but we see that the `tun_server.py` VPN server does not even capture any packets.

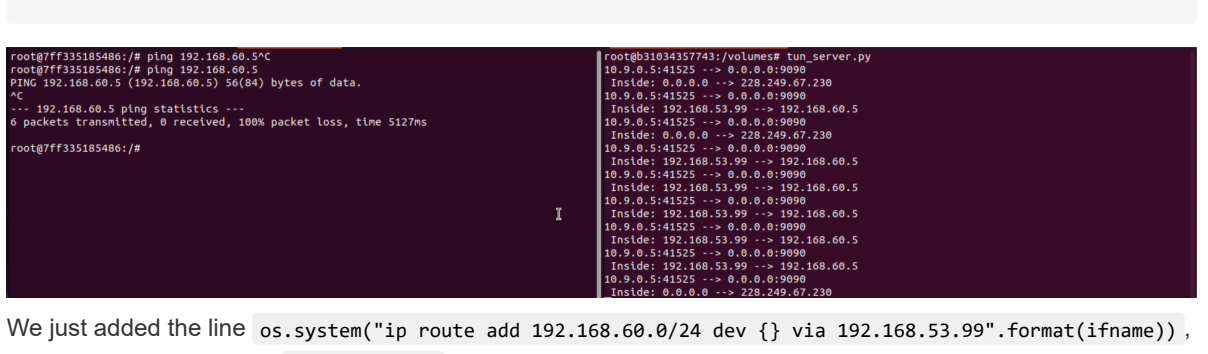


```
root@7ff335185486:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2025ms
root@7ff335185486:/#
```

```
root@b31034357743:/volumes# tun_server.py
```

   - We need to modifier the `tun_client.py` to add an interface for the `192.168.60.0/24` subnet to route packets through the TUN interface.
6. Therefore, as per the instructions in the lab, just modify the `tun_client.py` to:

```python
#!/usr/bin/env python3

import fcntl
import struct
import os
import time
from scapy.all import *


TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))
os.system("ip route add 192.168.60.0/24 dev {} via 192.168.53.99".format(ifname))


sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
SERVER_IP, SERVER_PORT = '10.9.0.11', 9090

while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        # Send the packet via the tunnel
        sock.sendto(packet, (SERVER_IP, SERVER_PORT))
```



```
root@7ff335185486:/# ping 192.168.60.5^C
root@7ff335185486:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
6 packets transmitted, 0 received, 100% packet loss, time 5127ms

root@7ff335185486:/#
```

```
root@b31034357743:/volumes# tun_server.py
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 220.249.67.230
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 220.249.67.230
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:41525 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 220.249.67.230
```

   - We just added the line `os.system("ip route add 192.168.60.0/24 dev {} via 192.168.53.99".format(ifname))`, then now we see that the `tun_server.py` on the VPN server is able to capture the ICMP echo request packets when we ping Host V.

Task 4: Setting up the VPN server

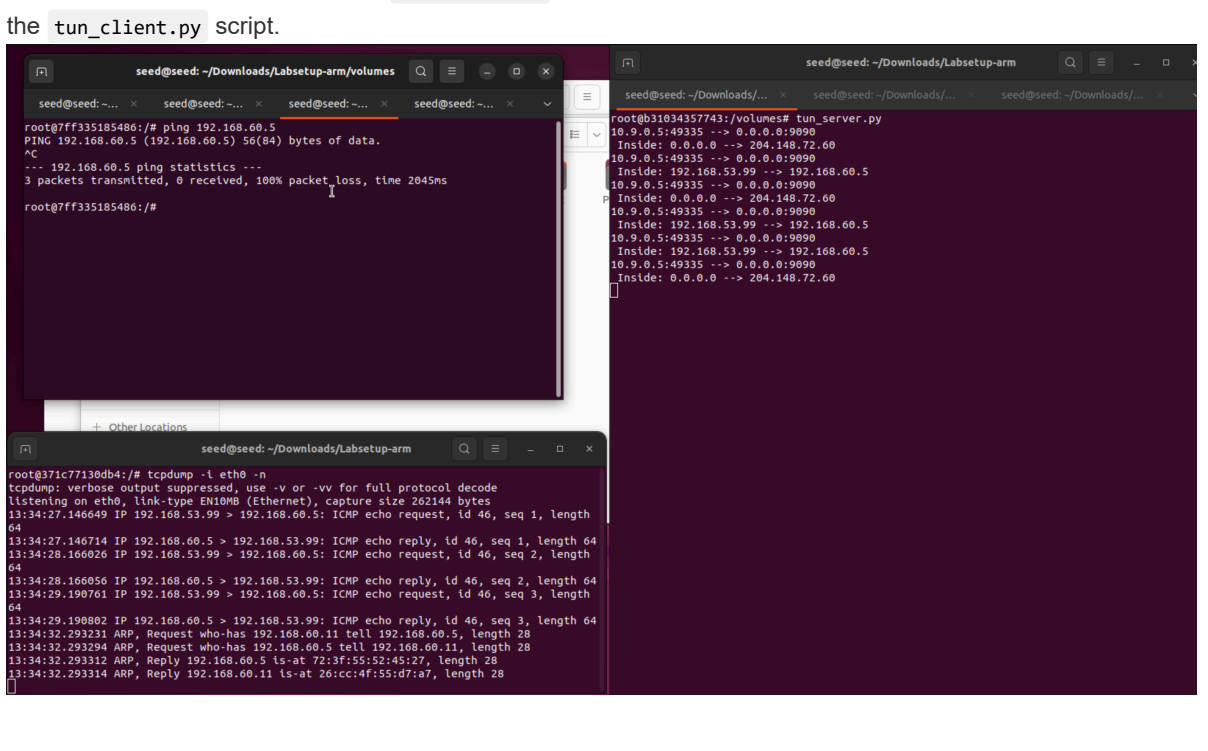
1. Based on the instructions of the manual, we need to modify the `tun_server.py` script to have the following features
   i. Create a TUN interface and configure it.
   ii. Get the data from the socket interface; treat the received data as an IP packet.
   iii. Write the packet to the TUN interface.
2. This is the modified `tun_server.py` script:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# 1.Create the tun interface and configure it.
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
# up the interface
os.system("ip addr add 192.168.53.11/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

# setup UDP socket and bind it to 0.0.0.0:9090
IP_A = "0.0.0.0"
PORT = 9090
sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

while True:
    # 2. Get data from the socket interface
    data, (ip,port) = sock.recvfrom(2048)
    print("{}:{} --> {}:{}".format(ip, port, IP_A, PORT))
    # treat as IP packet
    pkt = IP(data)
    print(" Inside: {} --> {}".format(pkt.src, pkt.dst))
    # 3. Write the packet to the TUN interface
    os.write(tun, bytes(pkt))
```

3. Now we run the modified `tun_server.py` on the VPN server, and the `tun_client.py` on Host U.

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# 1.Create the tun interface and configure it.
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
# up the interface
os.system("ip addr add 192.168.53.11/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))


# setup UDP socket and bind it to 0.0.0.0:9090
IP_A = "0.0.0.0"
PORT = 9090
sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

while True:
    # 2. Get data from the socket interface
    data, (ip,port) = sock.recvfrom(2048)
    print("{}:{} --> {}:{}".format(ip, port, IP_A, PORT))
    # treat as IP packet
    pkt = IP(data)
    print(" Inside: {} --> {}".format(pkt.src, pkt.dst))
    # 3. Write the packet to the TUN interface
    os.write(tun, bytes(pkt))
```

   - Then we ssh into Host U and run `ping 192.168.60.5` to ping Host V, we see that the `tun_server.py` on the VPN server is able to capture the ICMP echo request packets, and write them to the TUN interface on the VPN server, which then routes the packets to Host V.
   - This can be verified by running `tcpdump -i eth0 -n` on Host V, we see that the ICMP echo request packets are captured on Host V and come from `192.168.53.99` which is the TUN interface IP of Host U, which was setup in the `tun_client.py` script.



```
root@7ff335185486:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2045ms

root@7ff335185486:/#
```

```
root@b31034357743:/volumes# tun_server.py
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 204.148.72.60
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 204.148.72.60
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 192.168.53.99 --> 192.168.60.5
10.9.0.5:49335 --> 0.0.0.0:9090
 Inside: 0.0.0.0 --> 204.148.72.60
```

```
root@371c77130db4:/# tcpdump -i eth0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
13:34:27.146649 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 46, seq 1, length 64
13:34:27.146714 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 46, seq 1, length 64
13:34:28.166026 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 46, seq 2, length 64
13:34:28.166050 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 46, seq 2, length 64
13:34:29.190761 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 46, seq 3, length 64
13:34:29.190802 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 46, seq 3, length 64
13:34:32.293231 ARP, Request who-has 192.168.60.11 tell 192.168.60.5, length 28
13:34:32.293294 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
13:34:32.293312 ARP, Reply 192.168.60.5 is-at 72:3f:55:52:45:27, length 28
13:34:32.293314 ARP, Reply 192.168.60.11 is-at 26:cc:4f:55:d7:a7, length 28
```

Task 5: Handling traffic in both directions

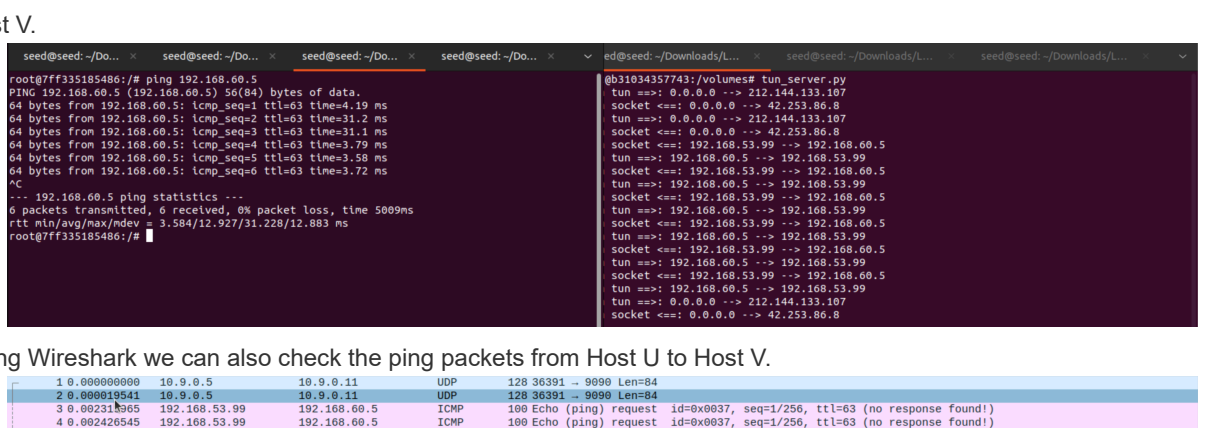
1. As per the lab instructions, we need to modify the `tun_server.py` and `tun_client.py` to handle traffic in both directions.
2. Modified `tun_server.py`:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# 1.Create the tun interface and configure it.
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
# up the interface
os.system("ip addr add 192.168.53.11/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))


# setup UDP socket and bind it to 0.0.0.0:9090
IP_A = "0.0.0.0"
PORT = 9090
HOST_U_IP = "10.9.0.5"
HOST_U_PORT = 9090
sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))
while True:

    # this will block until at least one interface is ready
    ready, _, _ = select.select([sock, tun], [], [])
    for fd in ready:
        if fd is sock:
            data, (HOST_U_IP, HOST_U_PORT) = sock.recvfrom(2048)
            pkt = IP(data)
            print("From socket <==: {} --> {}".format(pkt.src, pkt.dst))
            os.write(tun, bytes(pkt))
        if fd is tun:
            packet = os.read(tun, 2048)
            pkt = IP(packet)
            print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
            sock.sendto(packet, (HOST_U_IP, HOST_U_PORT))
```

3. Modified `tun_client.py`:

```python
#!/usr/bin/env python3
import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))
os.system("ip route add 192.168.60.0/24 dev {} via 192.168.53.99".format(ifname))

sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
VPN_SERVER_IP, VPN_SERVER_PORT = '10.9.0.11', 9090
while True:

    # this will block until at least one interface is ready
    ready, _, _ = select.select([sock, tun], [], [])
    for fd in ready:
        if fd is sock:
            data, (VPN_SERVER_IP, VPN_SERVER_PORT) = sock.recvfrom(2048)
            pkt = IP(data)
            print("From socket <==: {} --> {}".format(pkt.src, pkt.dst))
            os.write(tun, bytes(pkt))
        if fd is tun:
            packet = os.read(tun, 2048)
            pkt = IP(packet)
            print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
            sock.sendto(packet, (VPN_SERVER_IP, VPN_SERVER_PORT))
```

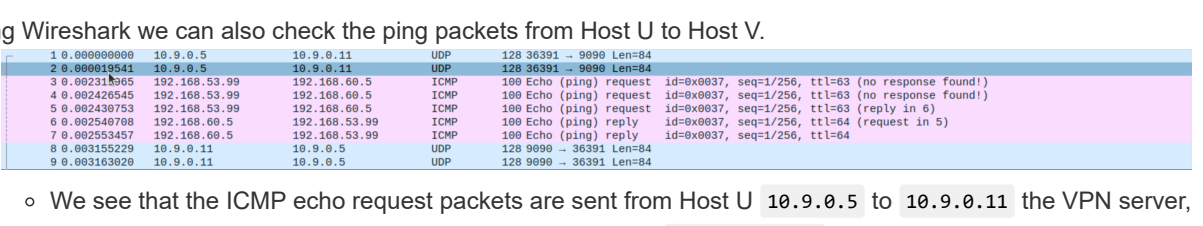
4. Now we run the modifier `tun_server.py` on the VPN server and `tun_client.py` on Host U.
5. Then we ssh into Host U and run `ping 192.168.60.5` to ping Host V, we see that the `tun_server.py` on the VPN server is able to capture the ICMP echo request packets, and write them to the TUN interface on the VPN server, which then routes the packets to Host V, and our ping command is able to receive ICMP echo reply packets from Host V.



```
root@7ff335185486:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=4.19 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=31.2 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=31.1 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=3.79 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=3.58 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=63 time=3.72 ms
^C
--- 192.168.60.5 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 3.584/12.927/31.228/12.883 ms
root@7ff335185486:/#
```

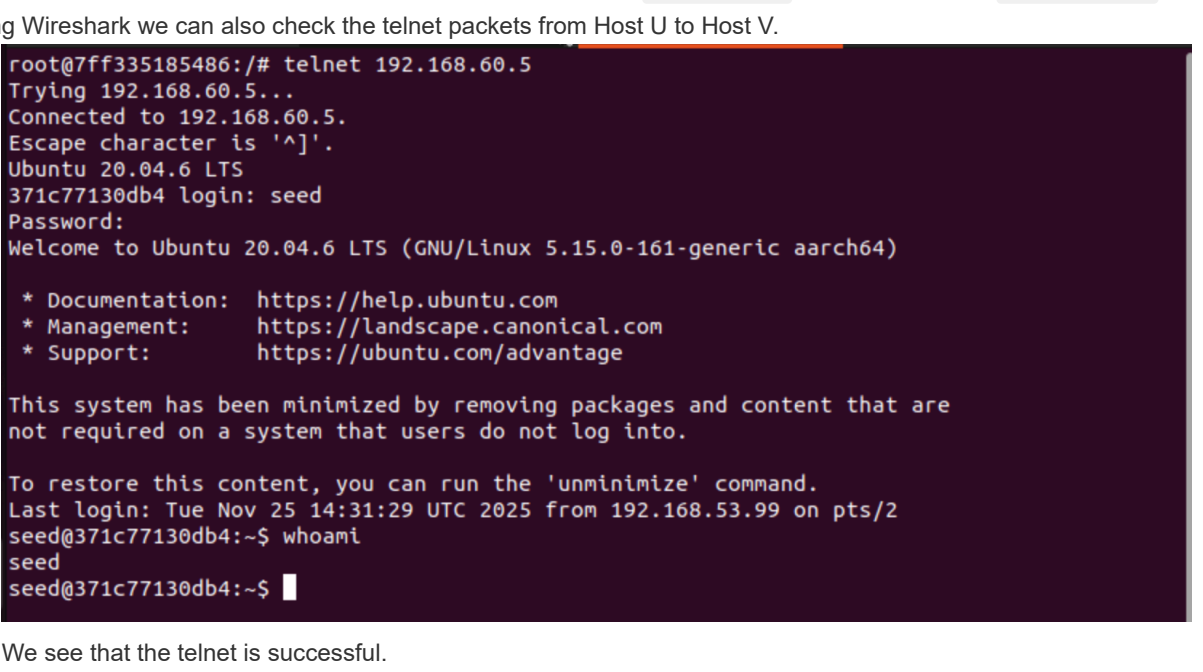
```
root@b31034357743:/volumes# tun_server.py
tun ==>: 0.0.0.0 --> 212.144.133.107
socket <==: 0.0.0.0 --> 42.253.86.8
tun ==>: 0.0.0.0 --> 212.144.133.107
socket <==: 0.0.0.0 --> 42.253.86.8
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
socket <==: 192.168.53.99 --> 192.168.60.5
tun ==>: 192.168.60.5 --> 192.168.53.99
tun ==>: 0.0.0.0 --> 212.144.133.107
socket <==: 0.0.0.0 --> 42.253.86.8
```

6. Using Wireshark we can also check the ping packets from Host U to Host V.



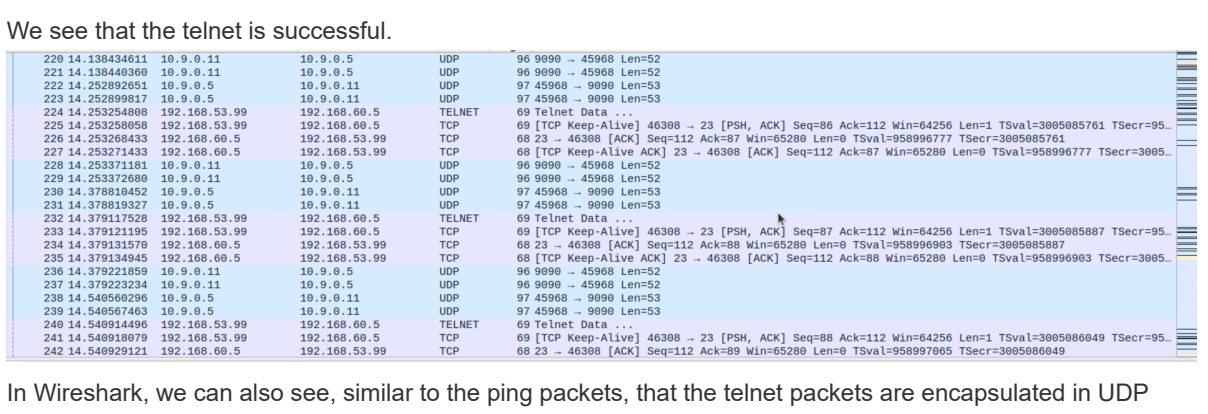
```
1 0.000000000  10.9.0.5       10.9.0.11      UDP   128 36391 → 9090 Len=84
2 0.000019541  10.9.0.5       10.9.0.11      UDP   128 36391 → 9090 Len=84
3 0.002313965  192.168.53.99  192.168.60.5   ICMP  100 Echo (ping) request  id=0x0037, seq=1/256, ttl=63 (no response found!)
4 0.002426545  192.168.53.99  192.168.60.5   ICMP  100 Echo (ping) request  id=0x0037, seq=1/256, ttl=63 (no response found!)
5 0.002430753  192.168.53.99  192.168.60.5   ICMP  100 Echo (ping) request  id=0x0037, seq=1/256, ttl=63 (reply in 6)
6 0.002540708  192.168.60.5   192.168.53.99  ICMP  100 Echo (ping) reply    id=0x0037, seq=1/256, ttl=64 (request in 5)
7 0.002553457  192.168.60.5   192.168.53.99  ICMP  100 Echo (ping) reply    id=0x0037, seq=1/256, ttl=64
8 0.003155229  10.9.0.11      10.9.0.5       UDP   128 9090 → 36391 Len=84
9 0.003163020  10.9.0.11      10.9.0.5       UDP   128 9090 → 36391 Len=84
```

   - We see that the ICMP echo request packets are sent from Host U `10.9.0.5` to `10.9.0.11` the VPN server, as UDP packets as they are encapsulated in UDP by the `tun_client.py` script.
   - The replies are also UDP packets from the VPN server `10.9.0.11` to Host U `10.9.0.5` and contain the ICMP echo reply packets.
   - We see that the ICMP echo request packets are sent from the VPN server in the internal network `192.168.53.99` to Host V `192.168.60.5`, as normal ICMP packets as they are written to the TUN interface by the `tun_server.py` script.
   - The replies are also normal ICMP packets from Host V `192.168.60.5` to the VPN server `192.168.53.99`.
7. Using Wireshark we can also check the telnet packets from Host U to Host V.



```
root@7ff335185486:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.6 LTS
371c77130db4 login: seed
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-161-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Nov 25 14:31:29 UTC 2025 from 192.168.53.99 on pts/2
seed@371c77130db4:~$ whoami
seed
seed@371c77130db4:~$
```

   - We see that the telnet is successful.

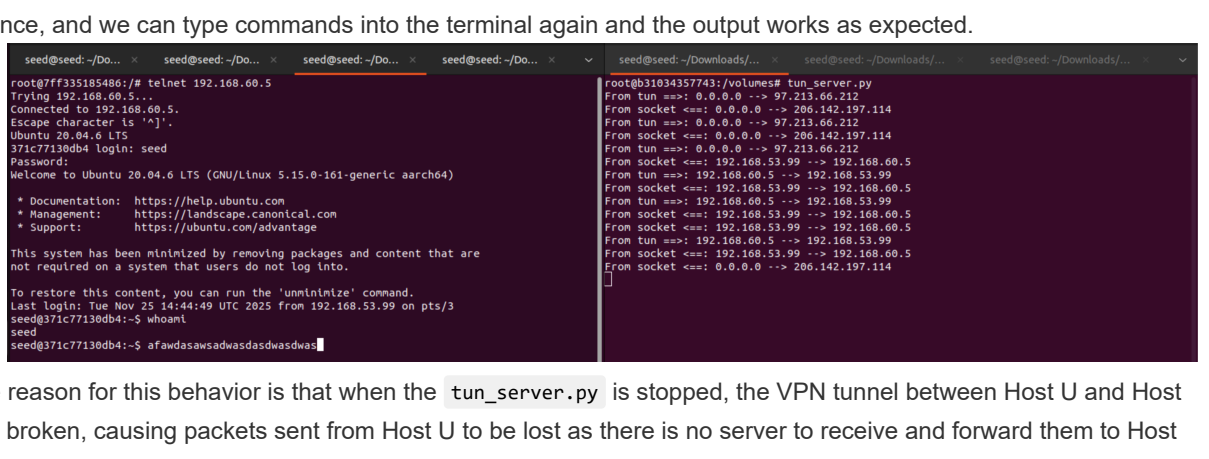


```
220 14.138434611  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
221 14.138440360  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
222 14.252892651  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
223 14.252899817  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
224 14.253254800  192.168.53.99  192.168.60.5   TELNET  69 Telnet Data ...
225 14.253258058  192.168.53.99  192.168.60.5   TCP     69 [TCP Keep-Alive] 46308 → 23 [PSH, ACK] Seq=86 Ack=112 Win=64256 Len=1 TSval=3005085761 TSecr=95…
226 14.253268433  192.168.60.5   192.168.53.99  TCP     68 23 → 46308 [ACK] Seq=112 Ack=87 Win=65280 Len=0 TSval=958996777 TSecr=3005085761
227 14.253271433  192.168.60.5   192.168.53.99  TCP     68 [TCP Keep-Alive ACK] 23 → 46308 [ACK] Seq=112 Ack=87 Win=65280 Len=0 TSval=958996777 TSecr=3005…
228 14.253371181  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
229 14.253372680  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
230 14.378810452  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
231 14.378819327  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
232 14.379117528  192.168.53.99  192.168.60.5   TELNET  69 Telnet Data ...
233 14.379121195  192.168.53.99  192.168.60.5   TCP     69 [TCP Keep-Alive] 46308 → 23 [PSH, ACK] Seq=87 Ack=112 Win=64256 Len=1 TSval=3005085887 TSecr=95…
234 14.379131570  192.168.60.5   192.168.53.99  TCP     68 23 → 46308 [ACK] Seq=112 Ack=88 Win=65280 Len=0 TSval=958996903 TSecr=3005085887
235 14.379134945  192.168.60.5   192.168.53.99  TCP     68 [TCP Keep-Alive ACK] 23 → 46308 [ACK] Seq=112 Ack=88 Win=65280 Len=0 TSval=958996903 TSecr=3005…
236 14.379221859  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
237 14.379223234  10.9.0.11      10.9.0.5       UDP     96 9090 → 45968 Len=52
238 14.540560296  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
239 14.540567463  10.9.0.5       10.9.0.11      UDP     97 45968 → 9090 Len=53
240 14.540914496  192.168.53.99  192.168.60.5   TELNET  69 Telnet Data ...
241 14.540918079  192.168.53.99  192.168.60.5   TCP     69 [TCP Keep-Alive] 46308 → 23 [PSH, ACK] Seq=88 Ack=112 Win=64256 Len=1 TSval=3005086049 TSecr=95…
242 14.540929121  192.168.60.5   192.168.53.99  TCP     68 23 → 46308 [ACK] Seq=112 Ack=89 Win=65280 Len=0 TSval=958997065 TSecr=3005086049
```

   - In Wireshark, we can also see, similar to the ping packets, that the telnet packets are encapsulated in UDP packets when sent from Host U `10.9.0.5` to the VPN server `10.9.0.11` through the `tun_client.py` script, and are normal TCP packets when sent from the VPN server on the internal network `192.168.53.99` to Host V `192.168.60.5`, through the `tun_server.py` script.

Task 6: Tunnel Breaking Experiment

1. When we run `telnet` from host U to host V, then we stop the `tun_server.py` on the VPN server, we see that the telnet terminal becomes unresponsive and nothing is displayed when we type anything into the terminal.
2. Then we restart both the `tun_server.py` on the VPN server and `tun_client.py` on Host U, and we see that the telnet session is restored, whatever we typed into the keyboard when the terminal was unresponsive shows up all at once, and we can type commands into the terminal again and the output works as expected.



```
root@7ff335185486:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.6 LTS
371c77130db4 login: seed
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-161-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Nov 25 14:44:49 UTC 2025 from 192.168.53.99 on pts/3
seed@371c77130db4:~$ whoami
seed
seed@371c77130db4:~$ afawdasawsadwasdasdwasdwas
```

```
root@b31034357743:/volumes# tun_server.py
From tun ==>: 0.0.0.0 --> 97.213.66.212
From socket <==: 0.0.0.0 --> 206.142.197.114
From tun ==>: 0.0.0.0 --> 97.213.66.212
From socket <==: 0.0.0.0 --> 206.142.197.114
From tun ==>: 0.0.0.0 --> 97.213.66.212
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From socket <==: 0.0.0.0 --> 206.142.197.114
```

3. The reason for this behavior is that when the `tun_server.py` is stopped, the VPN tunnel between Host U and Host V is broken, causing packets sent from Host U to be lost as there is no server to receive and forward them to Host V. This results in the telnet session becoming unresponsive since the packets cannot reach their destination, but the telnet packets are buffered on Host U's shell, then when the `tun_server.py` is restarted, the VPN tunnel is re-established, allowing the buffered packets to be sent to Host V all at once, restoring the telnet session, the buffered packets are sent in a burst to Host V, which is why they all appear at once in the terminal.